

Trabajo Práctico Integrador

PROGRAMACIÓN 2

Alumna: Couchot María Paz

Tecnicatura Universitaria en Programación - UTN

Docente Titular

Cinthia Rigoni

Docente Tutor

Marcos Vega

INDICE

● Alcance de Programación 2	1
● Información del proyecto	1
● Objetivos de aprendizaje	1
● Marco teórico	2
● Decisiones técnicas y arquitectura	2
● Dificultades y soluciones	4

Alcance de Programación 2

- Toda la interacción ocurre en consola mediante un menú.
- Se aplique POO (encapsulamiento, constructores, herencia, polimorfismo) según el UML.
- Se utilicen Colecciones para el almacenamiento y gestión de los objetos en memoria.
- Se implementen Interfaces para definir comportamientos comunes.
- Se aplique el Manejo de Excepciones para validar errores y evitar cierres inesperados del programa.

Información del Proyecto

- Modalidad: Aplicación de consola (menú).
- Lenguaje: Java 21.
- Almacenamiento: En memoria (Colecciones).
- Patrones/Capas: Entidades + Servicios/Lógica + UI (Main/Menu).

Objetivos de Aprendizaje

Al finalizar el TPI, el proyecto debe evidenciar que el estudiante puede:

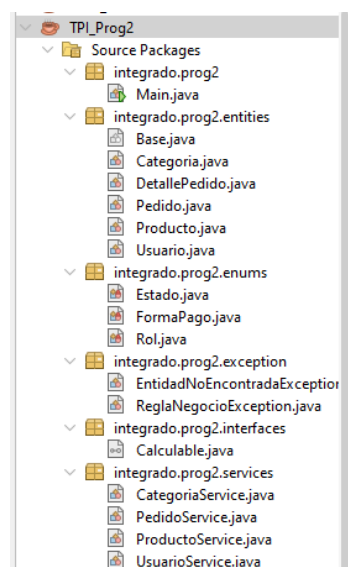
- Modelar correctamente clases y relaciones del UML (asociaciones 1..N y N..1).
- Aplicar herencia (clase base) y sobrescritura de métodos (toString, equals).
- Utilizar estructuras de datos dinámicas (Colecciones) para gestionar las entidades.
- Implementar Interfaces para estandarizar comportamientos (ej. Calculable).
- Gestionar errores creando y capturando Excepciones propias (por ejemplo, EntidadNoEncontradaException, StockInvalidoException).

MARCO TEÓRICO

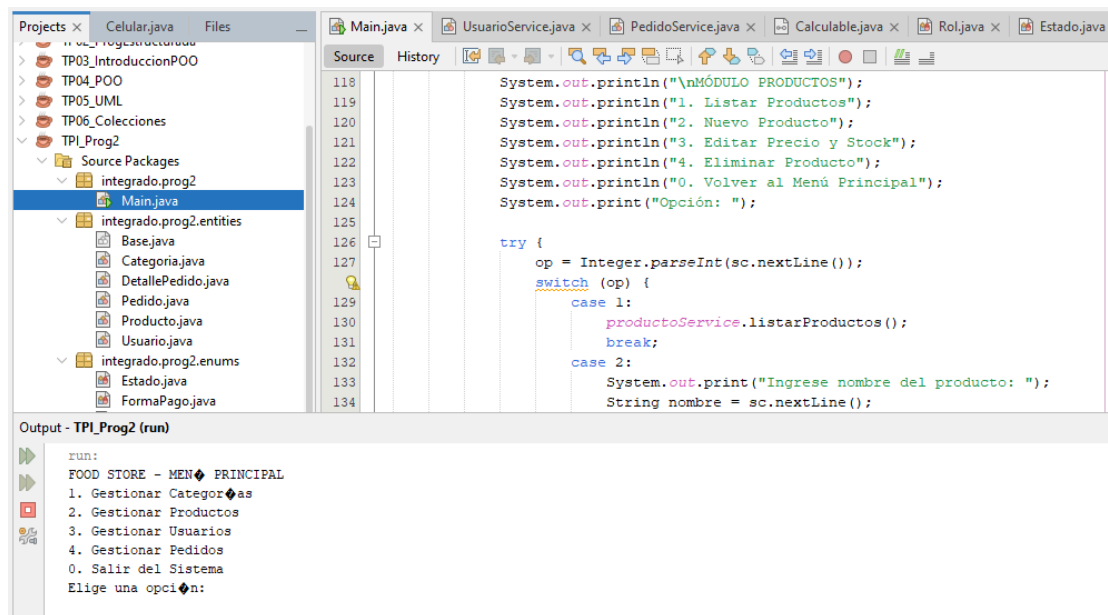
El desarrollo del sistema Food Store se fundamenta en los principios de la Programación Orientada a Objetos (POO) mediante el lenguaje Java. Se aplicaron los cuatro pilares esenciales del paradigma: la abstracción para modelar las entidades del dominio gastronómico, el encapsulamiento para proteger el estado interno de los objetos exponiendo únicamente métodos de acceso y modificación (getters and setters), la herencia centralizando atributos comunes como el identificador y el borrado lógico en una superclase, y el polimorfismo mediante la sobrescritura de métodos para la representación en cadena de texto. Ante la directiva de no utilizar bases de datos, la persistencia temporal en memoria se gestionó mediante colecciones dinámicas, específicamente la implementación de ArrayList, que permite una adaptación elástica frente a conjuntos de datos variables superando las limitaciones de los arreglos estáticos. Además, se estableció un contrato de comportamiento a través de una interfaz para el cálculo de totales garantizando un bajo acoplamiento, y se implementó una arquitectura defensiva utilizando tanto excepciones nativas de la clase Scanner como excepciones chequeadas personalizadas para resguardar las reglas de negocio en tiempo de ejecución

DECISIONES TÉCNICAS Y ARQUITECTURA

Para asegurar un diseño modular con alta cohesión y bajo acoplamiento, el proyecto se estructuró en distintos paquetes especializados separando las responsabilidades del sistema.



La capa de entidades agrupa el modelo de dominio donde todas las clases heredan de una base común, trabajando en conjunto con un paquete de enumeraciones para estandarizar valores fijos como los roles o estados y prevenir errores tipográficos en el código. Los contratos de comportamiento y el manejo de errores se aislaron en sus propios paquetes de interfaces y excepciones para facilitar su reutilización global. Toda la lógica de negocio y el control del estado de las colecciones se centralizaron en una capa de servicios, cumpliendo con el principio de responsabilidad única al encargarse exclusivamente de validar datos, aplicar reglas de negocio y gestionar los identificadores. Finalmente, la clase principal se configuró únicamente como la capa de vista para gestionar el menú y la interacción por consola, delegando toda la responsabilidad de operaciones y cálculos a los servicios correspondientes. En esta sección del documento se adjuntan las capturas de pantalla que evidencian la estructura de paquetes en el entorno de desarrollo y el menú principal en funcionamiento.



The screenshot displays an IDE with the following components:

- Projects Panel:** Shows a project named 'TPI_Prog2' with a package structure including 'integrado.prog2.entities' (containing Base.java, Categoria.java, DetallePedido.java, Pedido.java, Producto.java, Usuario.java) and 'integrado.prog2.enums' (containing Estado.java, FormaPago.java).
- Source Editor:** Displays the code for 'Main.java', which includes a menu system and a switch statement for handling user input. The code is as follows:

```
118 System.out.println("\nMÓDULO PRODUCTOS");
119 System.out.println("1. Listar Productos");
120 System.out.println("2. Nuevo Producto");
121 System.out.println("3. Editar Precio y Stock");
122 System.out.println("4. Eliminar Producto");
123 System.out.println("0. Volver al Menú Principal");
124 System.out.print("Opción: ");
125
126
127 try {
128     op = Integer.parseInt(sc.nextLine());
129     switch (op) {
130         case 1:
131             productoService.listarProductos();
132             break;
133         case 2:
134             System.out.print("Ingrese nombre del producto: ");
135             String nombre = sc.nextLine();
```
- Output Panel:** Shows the console output for 'TPI_Prog2 (run)', displaying the menu and the user's selection of option 1.

```
run:
FOOD STORE - MENÚ PRINCIPAL
1. Gestionar Categorías
2. Gestionar Productos
3. Gestionar Usuarios
4. Gestionar Pedidos
0. Salir del Sistema
Elige una opción:
```

A continuación, se evidencia el sistema en pleno funcionamiento resolviendo un flujo de negocio:

```
Módulo PEDIDOS
1. Listar Pedidos
2. Crear Nuevo Pedido (Venta)
3. Cambiar Estado de Pedido
0. Volver al Menú Principal
Opción: 2
Seleccione Cliente
Usuario [ID: 1, Nombre: Juan Perez, Mail: juan@mail.com, Rol: USUARIO]
ID Usuario: 1

Catálogo de Productos
Producto [ID: 1, Nombre: Cheeseburger, Precio: $8500,00, Stock: 10] | Categoría: Hamburguesas
ID Producto (0 para finalizar ticket): 1
Cantidad: 2
Producto agregado. Total parcial del pedido: $17000.0

Catálogo de Productos
Producto [ID: 1, Nombre: Cheeseburger, Precio: $8500,00, Stock: 8] | Categoría: Hamburguesas
ID Producto (0 para finalizar ticket): 0
Pedido cerrado. Total final: $17000.0
```

DIFICULTADES Y SOLUCIONES

Durante la implementación del sistema surgieron desafíos técnicos asociados a la interacción por consola y a la preservación de la integridad de los datos en la memoria. El primer inconveniente detectado fue la inconsistencia al limpiar el buffer del lector al intercalar ingresos numéricos y de texto, lo cual provocaba saltos de línea y errores de formato; esto se solucionó adoptando el patrón robusto recomendado por la cátedra, leyendo siempre la entrada completa como texto y utilizando las clases envolventes para su posterior conversión numérica dentro de bloques de protección contra excepciones. Otro desafío de diseño consistió en mantener actualizado el importe total del pedido de forma consistente al agregar nuevos detalles, lo que se resolvió aplicando el principio de encapsulamiento estricto haciendo que el propio objeto pedido invoque internamente a su método de cálculo cada vez que recibe una modificación en su lista de ítems. Por último, para evitar errores de referencias nulas en el sistema al eliminar registros vinculados históricamente, se implementó la estrategia de borrado lógico a través de un atributo booleano en la clase padre, lo que permitió que los servicios simplemente filtren de forma visual los elementos inactivos sin removerlos físicamente de las colecciones.

A continuación, se evidencia cómo el sistema se defiende en el caso de qué se ingresen datos inválidos:

```
pedido cancelado. Total final: $17000.0
```

```
Módulo PEDIDOS
```

1. Listar Pedidos
2. Crear Nuevo Pedido (Venta)
3. Cambiar Estado de Pedido
0. Volver al Menú Principal

```
Opción: 2
```

```
Seleccione Cliente
```

```
Usuario [ID: 1, Nombre: juan perez, Mail: juan@mail.com, Rol: USUARIO]
```

```
ID Usuario: 1
```

```
Catálogo de Productos
```

```
Producto [ID: 1, Nombre: Cheeseburger, Precio: $8500,00, Stock: 8] | Categoría: Hamburguesas
```

```
ID Producto (0 para finalizar ticket): 1
```

```
Cantidad: 30
```

```
Stock insuficiente. Quedan 8 unidades.
```